A PRELIMINARY PROJECT REPORT

ON

AI-BASED CUSTOMER SEGMENTATION AND PERSONALIZED WHATSAPP MARKETING PLATFORM

Submitted to the Savitribai Phule Pune University, Pune
in partial fulfillment of the requirements for the award of the degree of

Bachelor of Engineering in Information Technology

BY

Sarthak Gawari	**43133**
Yash Kachale	**43XXX**
Sushant Bhanwase	**43XXX**
Jaykumar Kale	**43XXX**

UNDER THE GUIDANCE OF

Mr. Sandeep R. Warhade



DEPARTMENT OF INFORMATION TECHNOLOGY
PUNE INSTITUTE OF COMPUTER TECHNOLOGY, PUNE – 411 043

2026–2027

SCTR's
PUNE INSTITUTE OF COMPUTER TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY



C E R T I F I C A T E

This is to certify that the project report entitled

AI-Based Customer Segmentation and Personalized WhatsApp Marketing Platform

submitted by

Sarthak Gawari	**43133**
Yash Kachale	**43XXX**
Sushant Bhanwase	**43XXX**
Jaykumar Kale	**43XXX**

is a bonafide work carried out under the supervision of Mr. Sandeep R. Warhade. The work is submitted in partial fulfillment of the requirements of Savitribai Phule Pune University for the award of the degree of Bachelor of Engineering in Information Technology.

Mr. Sandeep R. Warhade
Internal Project Guide

Dr. S. C. Dharmadhikari
Head, Department of Information Technology

[External Examiner]
External Examiner

Dr. S. T. Gandhe
Principal, PICT Pune

Date: _____ Place: Pune

DECLARATION

We declare that the project report entitled **AI-Based Customer Segmentation and Personalized WhatsApp Marketing Platform** is our original work carried out as part of the Bachelor of Engineering programme in Information Technology. The implementation, experiments, documentation, and analysis presented in this report were completed for the project repository identified in this work.

The report distinguishes verified results of the implemented batch MVP from proposed real-time research extensions. The synthetic verification dataset, source code, tests, and generated project outputs are cited where they support a claim. No result from an unimplemented event-driven system is presented as an achieved result.

Sarthak Gawari	**43133**
Yash Kachale	**43XXX**
Sushant Bhanwase	**43XXX**
Jaykumar Kale	**43XXX**

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Mr. Sandeep R. Warhade, for technical direction, constructive review, and encouragement throughout the project. The feedback received during planning, implementation, and evaluation helped us convert an initial marketing idea into a reproducible software system.

We thank Dr. S. C. Dharmadhikari, Head of the Department of Information Technology, and Dr. S. T. Gandhe, Principal, Pune Institute of Computer Technology, for providing the academic environment and infrastructure required for this work.

We acknowledge the foundational contributions of Hughes, MacQueen, Rousseeuw, Davies and Bouldin, Calinski and Harabasz, and other researchers whose work informs RFM analysis, clustering, and customer analytics. We also thank our families and colleagues for their patience and support during the development and testing of the platform.

Sarthak Gawari 43133

Yash Kachale 43XXX

Sushant Bhanwase 43XXX

Jaykumar Kale 43XXX

ABSTRACT

Customer data is often available as transaction history, but raw transactions do not directly explain which customers should receive a campaign or whether a customer should be contacted at all. This project presents an end-to-end platform for customer segmentation and personalized WhatsApp campaign recommendation. The implemented batch workflow validates CSV transaction data, isolates cancellations and duplicates, calculates customer-level Recency, Frequency, and Monetary (RFM) features, evaluates K-Means models, generates interpretable segment labels, and applies explicit consent, suppression, recency, and rolling frequency-cap checks before producing a campaign recommendation.

The repository's bundled synthetic verification dataset contains 57 raw records and 15 customers. The recorded execution retains 52 rows for analysis, isolates two cancellations, reports five corrupt rows, and removes one duplicate. With a fixed random seed of 42, the workflow selects three clusters using the highest silhouette score of 0.4562. The corresponding Davies–Bouldin index is 0.7150 and the Calinski–Harabasz score is 13.42. The governance output evaluates 15 customers, approves 12 and blocks three for consent, suppression, or frequency-cap reasons. The project includes a FastAPI backend, a React and TypeScript dashboard, a mock-by-default WhatsApp adapter, and an automated suite of 87 passing tests.

The project is complete as a batch MVP and demonstrates explainable analytical and policy behaviour. Persistent event processing, incremental clustering, live WhatsApp sending, and stability-aware real-time evaluation are future work rather than implemented results. This boundary is maintained so that the report remains reproducible and technically defensible.

Keywords: Customer segmentation, RFM analysis, K-Means clustering, marketing fatigue, consent-aware recommendation, WhatsApp, FastAPI, React, explainable AI.

Contents

List of Figures

List of Tables

ABBREVIATIONS

API	:	Application Programming Interface
CSV	:	Comma-Separated Values
DBI	:	Davies–Bouldin Index
DPDP	:	Digital Personal Data Protection
GDPR	:	General Data Protection Regulation
HTTP	:	Hypertext Transfer Protocol
JSON	:	JavaScript Object Notation
K-Means	:	K-Means clustering algorithm
ML	:	Machine Learning
ORM	:	Object-Relational Mapping
RFM	:	Recency, Frequency, Monetary
REST	:	Representational State Transfer
SQL	:	Structured Query Language
UCD	:	Use Case Diagram
UI	:	User Interface
WCSS	:	Within-Cluster Sum of Squares
XAI	:	Explainable Artificial Intelligence

1. Introduction

1.1 Background

Digital commerce systems generate large volumes of transaction records. Each record usually describes an invoice, a customer, a date, a quantity, and a unit price. The information is valuable because it reveals behavioural patterns, but it is difficult to use directly when a marketing team needs a concise answer to three questions: which customers are valuable, which customers are becoming inactive, and which customers can be contacted responsibly?

A common operational response is to send the same message to a broad audience. This approach is simple but ignores differences in purchase behaviour and can create marketing fatigue. It also creates a governance problem: a customer may have withdrawn consent, may be on a suppression list, or may already have received the maximum number of messages allowed in a rolling period. A useful system therefore has to combine behavioural analysis with explicit communication constraints.

This project develops a platform that addresses this combined problem. The analytical core uses RFM features and unsupervised clustering. The decision layer maps interpretable customer groups to campaign archetypes. The governance layer prevents an otherwise attractive recommendation from becoming an unauthorised outreach decision. The user interface exposes the process as a sequence of upload, analysis, segments, metrics, and recommendations views.

1.2 Motivation

RFM analysis is attractive for a project implementation because it is understandable to both technical and business audiences. Recency measures how recently a customer purchased, frequency measures repeated purchasing, and monetary value measures the amount spent. Together they form a compact behavioural representation without requiring personally intrusive attributes.

Clustering is useful when labelled outcomes are unavailable. A business may not have enough reliable historical campaign-response labels to train a supervised model. K-Means can still identify groups with similar behaviour, provided that data quality, scaling, cluster selection, and interpretation are documented. The resulting labels are descriptive rather than predictive.

WhatsApp is treated in this project as a delivery channel and preview surface, not as permission to contact customers automatically. The system uses a mock adapter by default. This

protects development and demonstration environments from accidental dispatch and makes the difference between a recommendation, a preview, and a delivered message explicit.

1.3 Problem Statement

Transaction data is frequently stored as a collection of uncleaned line items. Missing identifiers, invalid dates, cancellations, negative quantities, duplicate lines, and extreme monetary values can distort customer profiles. Even after cleaning, unsupervised clusters do not automatically translate into safe campaign decisions. A system is required that performs the following operations in a traceable order:

1. validate and clean transaction data;
2. construct customer-level RFM features;
3. select and explain a clustering configuration;
4. produce data-driven customer segment labels;
5. recommend a campaign archetype with supporting reasons; and
6. block recommendations that violate consent, suppression, or fatigue rules.

1.4 Research Question

The implemented project answers the engineering question: *Can a reproducible RFM and K-Means workflow provide explainable customer segments and policy-aware campaign recommendations through a usable web dashboard?*

The related future research question is: *Can a stability-aware decision layer improve real-time RFM updates while maintaining low latency and eligibility guarantees?* The second question is not claimed as answered because the event-driven extension has not been implemented or evaluated.

1.5 Objectives

The objectives of the completed project are:

1. Build a robust CSV ingestion and quality-control pipeline.
2. Calculate RFM features using clear business definitions.
3. Evaluate K-Means values of K using three internal metrics.

4. Generate dynamic segment labels from cluster statistics.
5. Implement explainable campaign recommendations.
6. Enforce consent, suppression, and rolling frequency-cap rules.
7. Provide REST endpoints for analysis and a connected React dashboard.
8. Provide a mock-by-default WhatsApp adapter and preview workflow.
9. Verify core services using automated tests.

1.6 Scope

The scope includes batch CSV ingestion, customer-level analytics, unsupervised segmentation, deterministic recommendations, policy checks, API endpoints, dashboard views, and component testing. The scope does not include production authentication, a complete multi-tenant API workflow, durable event queues, a public WhatsApp webhook, live message dispatch, or a claim of campaign conversion improvement.

1.7 Report Organization

Chapter 1 introduces the problem and scope. Chapter 2 discusses the domain and relevant technical background. Chapter 3 presents requirements and feasibility. Chapter 4 describes the system architecture. Chapter 5 explains the data pipeline and RFM feature engineering. Chapter 6 presents clustering and recommendation methodology. Chapter 7 covers implementation. Chapter 8 describes the dashboard and user workflow. Chapter 9 reports testing and verified results. Chapter 10 discusses security, privacy, ethics, and limitations. Chapter 11 presents deployment and operation. Chapter 12 concludes the completed work and identifies future research.

2. Domain and Technical Background

2.1 Customer Relationship Management

Customer relationship management systems collect interactions and transactions so that an organisation can understand customer needs and coordinate communication. A transaction record is useful for operational accounting, but a marketing decision requires derived features. The platform therefore treats raw records as evidence from which an auditable profile is calculated.

A segmentation system can support prioritisation, but segmentation is not equivalent to personal knowledge. A cluster describes similarity under the selected features and preprocessing choices. It does not explain a customer's motives, demographic identity, or future response. This distinction is maintained in the project by using descriptive labels and heuristic campaign scores rather than claiming a predicted probability of conversion.

2.2 RFM Analysis

RFM is a widely used behavioural framework for direct marketing. For a customer i, recency is the elapsed time since the most recent valid purchase, frequency is the number of distinct invoices, and monetary value is the sum of valid line totals. The measures are compact, explainable, and suitable for small datasets.

RFM is not automatically reliable. The reference date must be defined, cancellation rows must not inflate spend, duplicate records must not inflate frequency, and skewed monetary values can dominate distance-based clustering. The project makes each of these choices explicit.

2.3 Unsupervised Clustering

K-Means partitions feature vectors into K groups by repeatedly assigning each vector to its nearest centroid and updating centroids. Its objective function is:

$$J = \sum_{k=1}^{K} \sum_{x_i \in S_k} \|x_i - \mu_k\|^2, \tag{2.1}$$

where S_k is cluster k and μ_k is its centroid. K-Means is computationally practical, but the result depends on feature scale, the selected K, initialisation, and random seed.

The project reports silhouette, Davies–Bouldin, and Calinski–Harabasz indices. The silhouette score compares cohesion with separation and is maximised. The Davies–Bouldin index measures similarity to the most similar competing cluster and is minimised. The Calinski–Harabasz score compares between-cluster dispersion with within-cluster dispersion and is maximised. No single score is treated as proof of business value.

2.4 Marketing Fatigue and Consent

A recommendation should not override a customer's communication preferences. The implemented fatigue service applies hard checks: explicit consent must be true, suppression must be false, and the number of messages in the last 30 days must remain below the configured cap. Campaign-specific recency rules may additionally affect eligibility. The output includes both passed and failed checks so that a marketer can inspect the reason.

The default frequency cap is three messages in a rolling 30-day window. This is a project policy parameter, not a universal legal threshold. In production, the parameter would require organisational approval, regulatory review, and monitoring.

2.5 Explainable Artificial Intelligence

Explainability in this project means that each recommendation includes the segment label, RFM values, campaign type, heuristic priority score, policy status, and a natural-language rationale. The system does not claim that this explanation is a causal explanation. It is an audit-oriented description of the deterministic rules that produced the recommendation.

2.6 Messaging Integration

The WhatsApp integration follows an adapter design. A mock adapter returns a simulated message result for development. A live adapter skeleton can be configured only when explicit credentials and live-mode configuration are supplied. The project demonstration remains simulated. A preview is not a delivery receipt, and a recommendation is not a permission grant.

3. Requirements and Feasibility

3.1 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement
FR1	The system shall accept a transaction CSV through the upload endpoint.
FR2	The system shall validate required columns and common header aliases.
FR3	The system shall report rejected rows, cancellations, and duplicates.
FR4	The system shall calculate customer-level recency, frequency, and monetary values.
FR5	The system shall evaluate feasible K-Means cluster counts and report metrics.
FR6	The system shall generate descriptive labels from cluster statistics.
FR7	The system shall expose segment profiles and customer RFM results.
FR8	The system shall produce a campaign recommendation with supporting explanation.
FR9	The system shall block outreach when consent, suppression, or fatigue checks fail.
FR10	The system shall provide a simulated WhatsApp preview without live dispatch by default.
FR11	The dashboard shall display ingestion, metrics, segments, and recommendations.
FR12	The test suite shall cover the core services and policy boundaries.

3.2 Non-Functional Requirements

The system should be reproducible with a fixed seed and documented input. It should provide understandable error messages, separate computation from presentation, avoid leaking secrets into source files, and preserve a safe default for messaging. The API should return structured JSON responses and the dashboard should remain usable when the backend is unavailable by displaying a health status rather than silently presenting stale results.

3.3 Use Cases

The primary actor is a marketing analyst. The analyst uploads a CSV, reviews data-quality findings, runs analysis, inspects cluster metrics, examines segment profiles, filters recommendations, and opens a simulated message preview. A system administrator configures the environment and controls whether live mode is available. A future integration actor would submit transaction or delivery events, but that use case is outside the completed MVP.

Table 3.2: Primary use-case flow

Use case	Main flow
Upload data	Select file, send to API, display validation summary.
Run analysis	Start RFM and clustering, store process-local result.
Inspect segments	View segment size, medians, labels, and customer profiles.
Review recommendations	View campaign type, score, explanation, and policy checks.
Preview campaign	Open simulated WhatsApp message for an eligible customer.
Review health	Poll the API health endpoint and show service state.

3.4 Feasibility Analysis

3.4.1 Technical Feasibility

The project is technically feasible because the workload is small enough for pandas and scikit-learn, while FastAPI provides a clear service boundary. React and TypeScript provide a typed browser client without requiring a large frontend framework. The architecture leaves room for SQLite during development and PostgreSQL in a later deployment.

3.4.2 Operational Feasibility

The dashboard follows the analyst's natural workflow: data first, analysis second, interpretation third, and action last. The mock adapter reduces operational risk during demonstrations. Explanations and failed policy checks support review by users who may not inspect Python source code.

3.4.3 Economic Feasibility

The implementation uses open-source software and local development tools. A production deployment would introduce costs for hosting, database operations, observability, and WhatsApp Business API usage. Those costs are outside the completed local MVP.

3.4.4 Legal and Ethical Feasibility

The system is designed around consent and suppression checks. The bundled dataset is synthetic and uses customer identifiers such as ‘CUST001‘; no real phone numbers are required. A production system would need an approved lawful basis, retention policy, access control, and review of applicable privacy and messaging regulations.

4. System Architecture and Design

4.1 Architecture Overview

The active application contains a React frontend, a FastAPI backend, analytical services, policy services, and a mock messaging adapter. The current upload and analysis workflow is process-local through ‘backend/session.py‘. SQLAlchemy models and tenant helpers exist in the repository, but they are not wired into every active router. This distinction is important because the report describes what runs today rather than an intended production architecture.

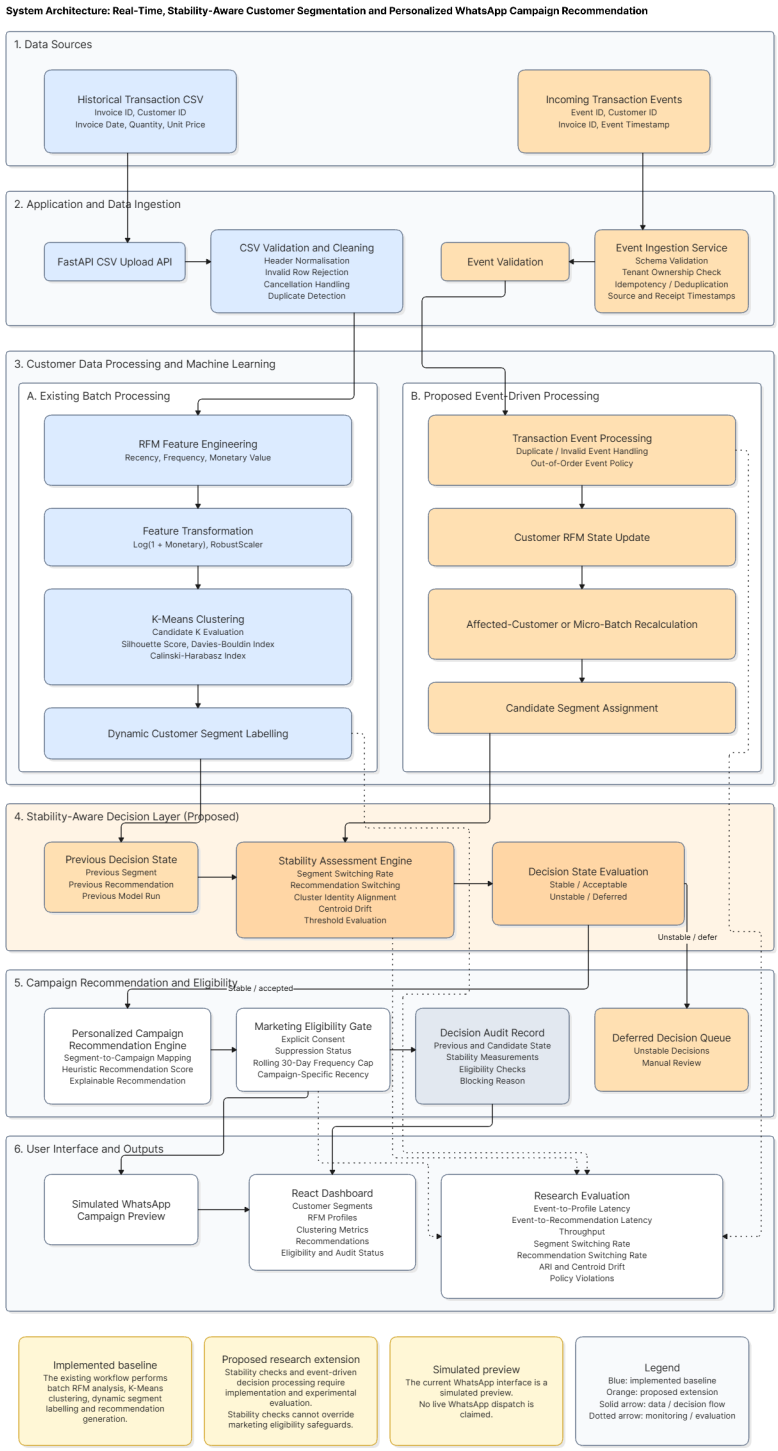


Figure 4.1: System Architecture: Real-Time, Stability-Aware Customer Segmentation and Personalized WhatsApp Campaign Recommendation. Blue denotes the implemented batch baseline; orange denotes the proposed extension.

4.2 Backend Modules

The ‘main.py‘ entry point creates the FastAPI application, configures CORS from environment settings, and registers the active routers. The health router provides a service status

response. The ingestion router accepts uploaded data and returns a cleaning summary. The analysis router executes RFM and clustering. Segment and recommendation routers expose computed results to the frontend.

The service layer keeps domain operations separate. The ingestion service is responsible for normalisation and validation. The RFM service aggregates customer features and assigns ordinal scores. The clustering service transforms features, evaluates K values, and creates dynamic labels. The fatigue service applies eligibility rules. The recommendation service combines segment characteristics with policy output.

4.3 Frontend Modules

The frontend is a React 18 and TypeScript application built with Vite. ‘App.tsx‘ controls view navigation. ‘Sidebar.tsx‘ provides navigation and backend health status. ‘UploadView.tsx‘ handles data ingestion and cleaning feedback. ‘SegmentsView.tsx‘ presents customer profiles and segment information. ‘MetricsView.tsx‘ presents clustering validation. ‘RecommendationsView.tsx‘ presents campaign decisions and a simulated WhatsApp preview.

4.4 Data Flow

The data flow is intentionally linear:

1. A CSV file is uploaded through the browser.
2. The API validates headers, parses values, removes invalid records, and identifies cancellations and duplicates.
3. Valid transactions are aggregated into customer-level RFM records.
4. Monetary value is transformed with $\log(1+M)$ and the feature matrix is robust-scaled.
5. Candidate K-Means models are evaluated and the highest silhouette score is selected.
6. Cluster statistics are converted to dynamic labels.
7. Recommendation rules map segment behaviour to a campaign archetype.
8. Consent, suppression, frequency, and recency checks determine eligibility.
9. The dashboard displays the result and may show a simulated preview.

4.5 API Design

Table 4.1: Implemented REST endpoints

Method	Path	Purpose
GET	‘/health‘	Return service status and timestamp.
POST	‘/api/v1/ingest/upload‘	Upload and clean a transaction CSV.
GET	‘/api/v1/ingest/status‘	Return current ingestion status.
POST	‘/api/v1/analysis/run‘	Execute RFM and clustering.
GET	‘/api/v1/analysis/rfm‘	Return customer RFM records.
GET	‘/api/v1/analysis/metrics‘	Return clustering metric values.
GET	‘/api/v1/segments‘	Return cohort profiles.
GET	‘/api/v1/recommendations‘	Return filterable recommendations.
POST	‘/api/v1/campaigns/preview‘	Return a simulated campaign preview.

4.6 Database and Persistence Boundary

SQLAlchemy models cover tenants, customers, transactions, campaigns, segments, and message events. However, the active MVP upload-to-analysis flow keeps its working state in memory. Therefore, a process restart removes the active analytical session. This is acceptable for a review-ready prototype but is not sufficient for production. The future persistence step must connect routers to database sessions, apply tenant filtering, and add restart and duplicate-event tests.

4.7 Design Principles

The design follows five principles. First, data-quality decisions are visible rather than silent. Second, model selection is reproducible. Third, segment names are generated from observed statistics instead of hard-coding cluster numbers. Fourth, policy checks are hard gates. Fifth, simulated communication is labelled as simulated. These principles reduce the gap between a technical demo and an auditable decision workflow.

5. Data Ingestion and Feature Engineering

5.1 Transaction Contract

The required transaction fields are invoice identifier, customer identifier, invoice date, quantity, and unit price. Optional metadata includes consent, suppression status, last-contact information, and recent-message count. The service supports common aliases such as ‘InvoiceNo‘, ‘CustomerID‘, ‘InvoiceDate‘, ‘Quantity‘, and ‘UnitPrice‘, then normalises them to internal names.

Table 5.1: Transaction fields and validation rules

Field	Type	Rule
invoice_id	String	Required; cancellation invoices begin with ‘C‘.
customer_id	String	Required; missing identifiers are rejected.
invoice_date	Date	Required; unparseable dates are rejected.
quantity	Numeric	Must be greater than zero for analytical rows.
unit_price	Numeric	Must be greater than zero.
consent_given	Boolean	Optional metadata for eligibility.
suppressed	Boolean	Optional suppression-list flag.
messages_last_30_days	Integer	Optional fatigue counter.

5.2 Validation and Cleaning

The ingestion service performs structural validation before aggregation. Rows with missing customer identifiers, invalid dates, non-positive quantities, and non-positive prices are dropped and recorded in an issue list. Invoice identifiers beginning with ‘C‘ are treated as cancellations. They are isolated in the cleaning summary and excluded from RFM aggregation. Exact duplicate lines are removed so the same invoice line does not inflate frequency or monetary value.

The bundled verification dataset intentionally includes valid records and controlled anomalies. This makes it possible to test the quality gate using a known expected output. The quality summary records both counts and reasons, which is more useful than returning only a cleaned dataframe.

5.3 RFM Definitions

For customer i, let t_{ref} be one day after the latest valid invoice date. Recency is:

$$R_i = (t_{ref} - \max_j t_{ij}).\text{days} \tag{5.1}$$

Frequency counts distinct invoice identifiers:

$$F_i = |\{\text{unique invoice identifiers belonging to } i\}|. \tag{5.2}$$

Monetary value is:

$$M_i = \sum_j q_{ij} p_{ij}, \tag{5.3}$$

where only valid, non-cancellation lines are included. The use of distinct invoices avoids treating multiple product lines from one invoice as multiple orders.

5.4 Ordinal RFM Scores

The raw RFM features are useful for clustering, while ordinal scores are useful for human interpretation. Values are divided into up to four rank-based bins. A smaller recency is better, so recency ranking is reversed. Larger frequency and monetary values receive larger scores. The composite score is:

$$Q_i = 0.3R_i^{score} + 0.3F_i^{score} + 0.4M_i^{score}. \tag{5.4}$$

The score is a ranking aid. It is not a learned response probability or a revenue forecast. If tied values prevent four quantile bins, a rank-based fallback preserves a valid score without inventing precision.

5.5 Feature Transformation

Retail monetary values are commonly right-skewed. A small number of high-value customers can dominate Euclidean distances if raw spend is used. The implementation applies:

$$M_i' = \log(1 + M_i). \tag{5.5}$$

The feature vector $[R_i, F_i, M_i']$ is then fitted with 'RobustScaler', which uses median and interquartile-range statistics. This choice reduces sensitivity to extreme values while preserving the relative ordering needed for interpretation.

5.6 Data Quality Output

Table 5.2: Verified ingestion output

Measure	Count
Original raw rows	57
Valid rows retained for analysis	52
Cancellations isolated	2
Dropped corrupt rows	5
Duplicate lines removed	1
Customer profiles represented	15

The controlled invalid records include a missing customer identifier, an unparseable date, a negative quantity, and a negative price. The generated output reports five dropped rows because the cleaning procedure also accounts for the duplicate and validation categories in its summary. The report uses the recorded artifact as the authoritative baseline rather than attempting to infer counts from a displayed CSV excerpt.

6. Clustering and Recommendation Methodology

6.1 K-Means Evaluation

The clustering service evaluates feasible values of K from two through a dataset-dependent upper bound. With 15 customer profiles, the recorded baseline sweep includes $K = 2$ through $K = 6$. The random seed is fixed at 42. The selected model is the candidate with the highest silhouette score, while the other indices remain available for comparison.

Table 6.1: Recorded clustering metric sweep

K	Silhouette	Davies–Bouldin	Calinski–Harabasz
2	0.4376	0.9209	10.25
3	**0.4562**	0.7150	13.42
4	0.4291	0.6505	15.59
5	0.3879	0.5166	13.53
6	0.3870	0.4231	13.90

The selection of $K = 3$ follows the documented silhouette criterion. The Davies–Bouldin score improves as K increases in this small sweep, but this does not by itself establish that five or six clusters are more meaningful. The selected value is therefore reported with all metrics and not justified through a single preferred number.

6.2 Dynamic Segment Labels

Cluster identifiers are arbitrary. Cluster zero in one run is not guaranteed to represent the same behaviour as cluster zero in another run. The label generator compares cluster medians with population medians and assigns descriptive names. This produces labels such as ‘Inactive Low-Value‘, ‘Loyal High-Value‘, and ‘Recent Frequent‘ based on observed behaviour.

Table 6.2: Verified segment profiles

Label	Size	Share	Recency	Frequency	Spend	Orientation
Inactive Low-Value	2	13.3%	685.0	1.5	INR 16.50	Targeted
Loyal High-Value	3	20.0%	2.0	3.0	INR 3,025.00	Targeted
Recent Frequent	10	66.7%	141.5	3.0	INR 235.48	Targeted

6.3 Campaign Mapping

Recommendations are rule-based. High-frequency or high-value groups are mapped to loyalty rewards; recent high-value behaviour can support an upsell; inactive groups can receive a win-back or re-engagement archetype. The exact mapping is inspectable in the recommendation service. It is not a supervised model and does not claim to estimate response probability.

6.4 Eligibility and Fatigue Rules

The recommendation output includes a policy audit object. The checks are evaluated in a clear order:

1. explicit consent must be present;
2. the customer must not be suppressed;
3. the rolling 30-day message count must be below three; and
4. any campaign-specific recency rule must pass.

A failed check produces a blocked recommendation with a reason. The analytical recommendation may still be visible for review, but it cannot be treated as permission to send.

Table 6.3: Policy decision behaviour

Check	Decision behaviour
Consent	False or missing consent blocks outreach.
Suppression	A suppression-list match blocks outreach.
Frequency cap	Three or more messages in the rolling 30-day period blocks outreach.
Recency	Campaign-specific timing conditions can block or alter eligibility.
Explanation	All checks and the blocking reason are returned to the user.

6.5 Recommendation Explanation

The explanation combines the customer identifier, segment label, latest purchase age, distinct invoice count, monetary total, recommended campaign, and eligibility status. Where engagement history is unavailable, the output states that limitation. The wording is intended to make the decision inspectable rather than to imply causal certainty.

7. Implementation

7.1 Technology Stack

Table 7.1: Implementation technology stack

Layer	Technology	Role
Frontend	React 18, TypeScript, Vite	Dashboard and user interaction.
Backend	Python, FastAPI, Uvicorn	REST API and application boundary.
Data	pandas, NumPy	Parsing and feature preparation.
Machine learning	scikit-learn	Scaling, K-Means, and metrics.
Validation	Pydantic	API schemas and structured data.
Persistence models	SQLAlchemy	Development database model boundary.
Testing	pytest	Automated service verification.

7.2 Backend Application Entry Point

The FastAPI application creates the service with a descriptive title and version. CORS origins are read from configuration. Routers are registered with the ‘/api/v1‘ prefix for versioned analytical endpoints. The root endpoint provides links to the API documentation and health endpoint. This makes the service straightforward to run locally with ‘uvicorn backend.main:app –reload –port 8000‘.

7.3 Ingestion Service

The ingestion service normalises column names, validates required fields, parses dates and numeric values, identifies cancellations, and returns a structured cleaning result. The service

does not hide data loss. Its output can be displayed directly by the upload view, allowing the analyst to understand why the usable population differs from the uploaded file.

7.4 RFM Service

The RFM service receives valid analytical transactions and groups them by customer. It calculates the latest purchase date, number of distinct invoices, and sum of valid line totals. It then generates ordinal scores and a composite ranking aid. The reference date is calculated from the data so that a reproducible run does not depend on the date on which a developer happens to execute the code.

7.5 Clustering Service

The clustering service prepares transformed features, evaluates candidate K values, selects the best silhouette candidate, and returns assignments and segment profiles. It handles small customer populations with validation errors or bounded candidate ranges. Dynamic labels use cluster statistics rather than relying on an unstable cluster number.

7.6 Recommendation and Fatigue Services

The recommendation service uses segment and RFM information to assign an archetype, priority score, explanation, and audit fields. The fatigue service supplies eligibility decisions. Keeping these responsibilities separate allows the project to test recommendation scoring independently of the communication policy.

7.7 WhatsApp Adapter

The adapter interface separates message construction from provider-specific transport. The mock implementation returns a simulated result and is safe for demonstrations. The live implementation requires explicit configuration. The active dashboard preview is a UI simulation and does not claim to create a persistent message event or send a live WhatsApp message.

7.8 Selected Pseudocode

Listing 7.1: Batch analysis workflow

```
raw = upload.read()
cleaned, summary = ingest_and_validate(raw)
rfm = calculate_rfm(cleaned)
features = prepare_features(rfm)
model, metrics = select_kmeans(features, random_seed=42)
segments = label_clusters(rfm, model.labels_)
recommendations = recommend_campaigns(rfm, segments)
for recommendation in recommendations:
    recommendation.audit = check_consent_suppression_and_fatigue(recommendation)
return summary, metrics, segments, recommendations
```

7.9 Error Handling

The application reports malformed files and insufficient customer populations as structured errors. The service avoids returning a misleading cluster result when there are not enough distinct customers. At the frontend, API health is displayed so the user can distinguish an unavailable backend from an empty analytical result.

8. Dashboard and User Experience

8.1 Dashboard Workflow

The dashboard is organised around the analyst's decision sequence. The upload view establishes data quality. The metrics view explains model selection. The segments view describes behavioural cohorts. The recommendations view connects the cohort to an action and displays policy checks. A sidebar provides navigation and backend health information.

8.2 Data Upload View

The upload view accepts a CSV file and displays the returned cleaning summary. It exposes original rows, retained rows, cancellations, dropped records, and duplicates. This view makes the quality gate a visible part of the workflow rather than an invisible preprocessing step.

8.3 Metrics View

The metrics view presents the selected K and the clustering indices. A user can see why three clusters were selected under the silhouette rule and can compare it with other tested values. The view is intended to support a technical review, not to imply that an internal metric directly measures campaign success.

8.4 Segments View

The segments view presents labels, cohort sizes, and customer-level RFM data. A label such as 'Loyal High-Value' is accompanied by the underlying statistics that generated it. This supports a conversation between an analyst and a business user about what the segment means.

8.5 Recommendations View

The recommendations view displays campaign archetypes, priority scores, explanations, eligibility outcomes, and blocking reasons. The simulated WhatsApp preview is available only

as a demonstration surface. It includes a visible simulated status so that a user cannot mistake the preview for a provider delivery receipt.

8.6 Usability Considerations

The interface uses a dark visual design system with clear panels and status indicators. Operational controls are grouped by task. Error and blocked states are shown as first-class outcomes. The dashboard does not require the user to know the internal Python service names to understand the data-quality summary or recommendation rationale.

8.7 Accessibility and Responsible Display

A production UI should add full keyboard testing, screen-reader labels, colour-contrast verification, and pagination for larger datasets. The MVP keeps customer IDs synthetic and does not display real phone numbers. A live deployment would require masking and role-based access to personally identifiable information.

9. Testing and Evaluation

9.1 Testing Strategy

Testing is organised by service boundary. Ingestion tests cover aliases, missing values, invalid dates, non-positive numbers, cancellations, duplicates, and summaries. RFM tests compare calculations against hand-verified formulas. Clustering tests cover valid and invalid population sizes, metric ranges, seed reproducibility, and dynamic labels. Fatigue tests cover consent, suppression, frequency cap, and recency decisions. Recommendation tests cover mapping, scores, explanations, and blocked states.

9.2 Automated Test Result

The generated project output records 87 collected tests with 87 passing in 2.38 seconds. The tests cover the implemented core services. This is evidence of component behaviour and regression protection; it is not evidence that the incomplete persistent, multi-tenant, real-time, or live WhatsApp paths are production-ready.

Table 9.1: Recorded automated test result

Measure	Result
Collected tests	87
Passed	87
Failed	0
Recorded duration	2.38 seconds
Coverage areas	ingestion, RFM, clustering, fatigue, recommendations

9.3 Data Quality Evaluation

The test dataset contains deliberately malformed and duplicate records. The expected output is 57 raw rows, 52 retained rows, two isolated cancellations, five dropped rows, one duplicate, and 15 customer profiles. These counts provide a discriminating check that the ingestion service is applying the intended quality rules rather than simply accepting every row.

9.4 RFM Verification

RFM tests verify recency against the reference date, frequency against distinct invoice identifiers, and monetary value against the sum of quantity multiplied by unit price. They also verify that cancellation lines do not increase monetary value. The generated output records population medians of 144.0 days for recency, 3.0 invoices for frequency, and INR 253.46 for monetary value.

9.5 Clustering Verification

The fixed seed provides reproducibility for the recorded baseline. The sweep selects $K = 3$ by silhouette score, with 0.4562 as the highest recorded value. The tests also exercise edge cases where a clustering metric is undefined or where the customer population is too small to support the requested range.

9.6 Policy Verification

Policy tests verify that false consent blocks outreach, suppression blocks outreach, and a message count of three reaches the default cap. They also verify that the output includes an explanation and a reason. These are safety invariants: a higher heuristic score must not override a failed policy check.

9.7 Integration Verification

The API can be started locally with FastAPI and the dashboard with Vite. The health endpoint returns service status and a timestamp. The generated project evidence also covers isolated adapter and integration components, but the report does not overstate them as a complete persistent provider integration.

9.8 Evaluation Boundaries

No live campaign conversion, delivery rate, latency benchmark, throughput benchmark, ARI value, centroid drift value, or recommendation-switching result is claimed. These values would require implementation of the proposed event path and a controlled evaluation protocol. The absence of such claims is a strength of the evidence boundary, not a missing result to be filled with assumptions.

10. Verified Results and Discussion

10.1 Dataset Description

The bundled dataset is a small synthetic verification dataset located at `backend/data/example_transac` It is designed to exercise common retail data-quality cases and does not represent a statistically general retail population. The latest valid date produces a reference date of 2024-10-06.

10.2 Population Results

Fifteen customer profiles are represented after cleaning. The recorded RFM table includes recent frequent customers, high-value customers, inactive customers, and low-spend customers. This range is sufficient to demonstrate the behaviour of the pipeline and the recommendation rules, while remaining too small for claims about general marketing effectiveness.

Table 10.1: Representative verified RFM profiles

Customer	Recency	Frequency	Monetary (INR)	Composite
CUST001	3	5	253.46	3.20
CUST002	2	8	654.50	4.00
CUST005	1	3	52,000.00	3.40
CUST006	371	2	28.00	1.00
CUST011	999	1	5.00	1.00
CUST012	183	2	3,025.00	2.50
CUST014	2	4	1,849.00	4.00

10.3 Cluster Results

The selected three-cluster solution gives an interpretable population split. The largest group is Recent Frequent with 10 customers. Loyal High-Value contains three customers and Inactive Low-Value contains two. The labels are generated from medians, so they should be understood as a summary of this run rather than as permanent customer identities.

10.4 Governance Results

The governance matrix approves 12 of 15 customers and blocks three. CUST009 is blocked because the frequency cap is reached. CUST003 is blocked because consent is absent. CUST008 is blocked because the customer is suppressed. This result demonstrates that the policy layer is capable of producing different decisions for customers who may otherwise appear in the same analytical workflow.

Table 10.2: Verified governance summary

Decision	Count	Share
Approved for simulated outreach	12	80.0%
Blocked by policy rules	3	20.0%
Total analysed customers	15	100.0%

10.5 Discussion of Findings

The results show that a small and inspectable pipeline can turn transaction line items into useful analytical cohorts and policy-aware recommendations. The cleaning summary is particularly important because it makes the difference between uploaded records and analytical records visible. The K-Means sweep shows that selection criteria matter: a larger number of clusters does not automatically produce a better silhouette score.

The results do not show that a loyalty campaign will increase revenue. They also do not show that a customer labelled Recent Frequent will respond to a WhatsApp message. The recommendation score ranks records according to deterministic project rules. This limitation is stated directly in the dashboard explanation and in this report.

10.6 Comparison with Project Objectives

All objectives for the completed batch MVP have been addressed. Ingestion, RFM, clustering, recommendations, fatigue rules, APIs, dashboard views, mock adapter behaviour, and automated tests are implemented. Objectives related to live sending, persistent event processing, and real-time stability remain future work and are not included in the completion claim.

11. Security, Privacy, Ethics, and Limitations

11.1 Security Controls

Configuration is supplied through environment variables rather than hard-coded credentials. The mock WhatsApp adapter is the default. The API has a health endpoint but the current MVP does not provide complete authentication and authorisation. Production deployment must add identity verification, role-based access, tenant-scoped queries, request limits, structured audit logs, secure secret storage, and transport security.

11.2 Privacy Controls

Consent and suppression are represented as explicit fields and checked before outreach eligibility. Customer IDs in the bundled data are synthetic. A production system must minimise identifiers, mask phone numbers in UI and logs, limit retention, encrypt sensitive data, and support correction and deletion workflows where required.

11.3 Ethical Use

Segmentation can create unfair or opaque treatment when used without review. The project limits itself to transaction behaviour and does not include protected attributes. Even so, a business user should review campaign rules and monitor whether groups are being systematically excluded or over-contacted. An explanation should support review, not remove human accountability.

11.4 Limitations of the Dataset

The 57-row dataset is synthetic and small. It is appropriate for verification but not for estimating population-level performance. Consent and engagement metadata are test fields. The report therefore does not claim external validity, campaign lift, or generalisation to a large retailer.

11.5 Limitations of the Architecture

The active workflow is process-local. A restart loses the working analysis session. SQLAlchemy models are present but not fully wired into the active routers. There is no public webhook endpoint, no durable campaign event store, and no complete authentication layer. The frontend preview does not invoke a real provider.

11.6 Limitations of the Model

K-Means assumes a distance-based structure and requires a selected K. Internal metrics do not measure marketing effectiveness. Dynamic labels can change across runs as population medians change. A heuristic priority score is not a response model. These limitations motivate future stability, calibration, and outcome-measurement work.

12. Deployment and Operation

12.1 Local Deployment

The backend can be run from the project root using:

```
python -m venv venv
.\\venv\\Scripts\\Activate.ps1
pip install -r backend\\requirements.txt
uvicorn backend.main:app --reload --port 8000
```

The frontend can be run from the ‘frontend‘ directory using:

```
npm install
npm run dev
```

The dashboard is served at ‘http://localhost:5173‘, the API at ‘http://localhost:8000‘, the API documentation at ‘/docs‘, and the health endpoint at ‘/health‘.

12.2 Configuration

The project uses environment configuration for database URLs, allowed origins, and WhatsApp mode. Live mode must remain false unless verified provider credentials and an approved operational process exist. Secrets must not be committed to source control.

12.3 Operational Runbook

An analyst should first confirm the API health indicator, upload a known CSV, inspect the cleaning summary, run analysis, inspect metric values, review segments, and then review blocked recommendations before opening a preview. The analyst should treat a simulated result as a demonstration only. If the dataset changes, the reference date, row counts, metrics, and segment profiles should be recorded again.

12.4 Monitoring Requirements

A production service should monitor upload failures, invalid-row rates, analysis duration, cluster metric changes, recommendation counts, blocked-policy counts, and adapter failures.

It should alert when the distribution of input data changes sharply or when consent and suppression data are unavailable. Monitoring should preserve privacy by aggregating identifiers wherever possible.

12.5 Reproducibility Checklist

A reproducible run should record the dataset name and hash, raw and retained row counts, cancellation and duplicate counts, reference date, transformation settings, scaler, K range, random seed, package versions, machine details, and generated metrics. If an external dataset such as UCI Online Retail is used, its source, download date, checksum, licence, and preprocessing manifest must also be recorded.

13. Future Research and Extension

13.1 Real-Time Research Direction

The planned research extension adds an event-driven or micro-batch path alongside the completed batch baseline. A transaction event would be validated, assigned an idempotency key, associated with a tenant, applied to the affected customer, and evaluated for a candidate segment and recommendation. A stability layer would compare the candidate state with the prior state before accepting a material change.

13.2 Proposed Stability Measures

For customer i, a segment switch indicator between runs t and $t+1$ is:

$$\delta_i^{seg}(t) = \mathbb{1}[a_i(t) \neq a_i(t+1)]. \tag{13.1}$$

The segment switching rate is:

$$S_{seg} = \frac{1}{n}\sum_{i=1}^{n} \delta_i^{seg}(t). \tag{13.2}$$

A similar rate can measure recommendation switching. Aligned centroid drift can be reported as:

$$D_{centroid} = \frac{1}{K}\sum_{k=1}^{K} \|\mu_k(t+1) - \mu_k(t)\|_2. \tag{13.3}$$

These definitions are proposed measures, not results from the current project.

13.3 Proposed Experimental Conditions

Future work should compare three conditions: the current batch baseline, event-driven updates without stability checks, and event-driven updates with stability checks. Replay of historical transactions may be used as a controlled simulation, but it must not be described as a live integration. Candidate strategies include affected-customer recomputation and ‘MiniBatchK-Means‘.

13.4 Future Acceptance Criteria

The extension should satisfy the following criteria before results are reported: duplicate events do not change totals; out-of-order events are handled according to a documented policy; tenant ownership is checked before mutation; previous and candidate decisions are recorded; unstable decisions are deferred rather than dispatched; and the batch baseline remains runnable for comparison.

13.5 Additional Integrations

The repository contains isolated components for tenant helpers, adapters, integration parsers, and a contextual bandit. These should only be included in the research evaluation after they are connected to the active workflow, tested through the API boundary, and supplied with durable feedback or event data. Isolated component tests are not sufficient evidence of an integrated production path.

14. Conclusion

14.1 Summary

This project implemented an end-to-end batch platform for customer segmentation and personalised WhatsApp campaign recommendation. It validates transaction data, calculates RFM profiles, evaluates K-Means clustering, generates dynamic labels, applies marketing governance rules, and exposes the outcome through FastAPI and React. The mock-by-default adapter supports safe demonstration without claiming live delivery.

The verified baseline uses 57 synthetic rows and 15 customers. It retains 52 rows, isolates two cancellations, reports five dropped rows, and removes one duplicate. The selected K-Means solution has three clusters with a silhouette score of 0.4562, a Davies–Bouldin index of 0.7150, and a Calinski–Harabasz score of 13.42. The governance output approves 12 customers and blocks three for explicit policy reasons. The core suite records 87 passing tests.

14.2 Contribution

The main contribution is an auditable combination of behavioural segmentation and communication governance. The system does not treat an analytical score as permission to contact a person. It exposes data-quality decisions, cluster metrics, recommendation reasoning, and blocked states in one workflow. This makes the prototype useful for technical demonstration, review, and future experimentation.

14.3 Final Status

The batch MVP is complete and review-ready. Real-time ingestion, stability-aware decisions, persistent webhook handling, production authentication, live WhatsApp delivery, and campaign-effectiveness measurement remain future work. Maintaining this boundary keeps the report aligned with the actual repository and prevents proposed research from being presented as completed evidence.

A. Sample API Responses

A.1 Health Response


```
{
  "status": "ok",
  "timestamp": "2026-10-26T09:28:42.413876Z",
  "service": "AI Customer Segmentation API",
  "version": "0.1.0"
}
```


A.2 Representative Recommendation Fields


```
{
  "customer_id": "CUST002",
  "segment_label": "Recent Frequent",
  "campaign_type": "loyalty_reward",
  "eligible": true,
  "consent_given": true,
  "suppressed": false,
  "messages_last_30_days": 1,
  "explanation": "RFM values and all fatigue checks are shown..."
}
```

B. Project Traceability Matrix

Table B.1: Requirement traceability

Requirement	Implementation	Evidence
CSV ingestion	‘backend/services/ingestion.py‘ and upload router	Ingestion tests and generated output.
RFM calculation	‘backend/services/rfm.py‘	Hand-verified RFM tests.
Clustering	‘backend/services/clustering.py‘	Metric sweep and clustering tests.
Fatigue policy	‘backend/services/fatigue.py‘	Consent, suppression, cap tests.
Recommendations	‘backend/services/recommendations.py‘	Mapping and explanation tests.
Dashboard	‘frontend/src/views/‘	Connected development UI.
Mock messaging	‘backend/adapters/whatsapp.py‘	Simulated adapter tests.

C. References

[1] A. Hughes, *Strategic Database Marketing*, Probus Publishing, 1994.

[2] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1967.

[3] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987.

[4] D. L. Davies and D. W. Bouldin, "A cluster separation measure," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-1, no. 2, pp. 224–227, 1979.

[5] T. Calinski and J. Harabasz, "A dendrite method for cluster analysis," *Communications in Statistics*, vol. 3, no. 1, pp. 1–27, 1974.

[6] M. Chen, A. Sain, and K. Guo, "Data mining for the online retail industry: Customer segmentation and market basket analysis," *Journal of Database Marketing and Customer Strategy Management*, 2012.

[7] P. S. Fader, B. G. S. Hardie, and K. L. Lee, "RFM and CLV: Using iso-value curves for customer base analysis," *Journal of Marketing Research*, vol. 42, no. 4, pp. 415–430, 2005.

[8] A. Bifet and R. Kirkby, *Data Stream Mining: A Practical Approach*, 2009.

[9] C. C. Aggarwal et al., "A framework for clustering evolving data streams," in *Proceedings of VLDB*, 2003.

[10] L. Hubert and P. Arabie, "Comparing partitions," *Journal of Classification*, vol. 2, pp. 193–218, 1985.

[11] M. Godfrey, M. Seiders, and G. Voss, "Enough is enough: The fine line in executing multichannel relational communication," *Journal of Marketing*, 2011.

[12] V. Kumar and W. Reinartz, *Creating Enduring Customer Value*, Springer, 2016.

[13] P. C. Verhoef, P. K. Kannan, and J. J. Inman, "From multi-channel retailing to omni-channel retailing," *Journal of Retailing*, vol. 91, no. 2, pp. 174–181, 2015.

[14] FastAPI Documentation, "FastAPI framework, high performance, easy to learn, fast to code," available at `https://fastapi.tiangolo.com/`.

[15] Scikit-learn Documentation, "Clustering and preprocessing," available at `https://scikit-learn.org/`.

D. Repository Evidence Used

The implementation and results in this report are based on the project repository's backend services, frontend views, automated tests, 'docs/STATUS.md', 'docs/ARCHITECTURE.md', `docs/PROJECT_OUTPUTS.md`, and the bundled synthetic transaction file. The report intentionally excludes unsupported historical K=4 figures and does not claim results for the proposed real-time extension.